ALEXANDRE Ewan, Groupe 1
09/10/2026

**Projet : Gestion de collocation avec Flutter et Supabase**

## Table des matières

1) Jalon 1 : ..... 1
1.1) Questions : ..... 1
1.2) Diagramme BDD + Script SQL ..... 2
1.3) Matrice des autorisations (RLS) et découpage des tâches ..... 5
1.4) Graphe de navigation et maquettes papiers ..... 6
2) Jalon 2 : ..... 8
2.1) Authentification et sélection de la colocation ..... 8
2.2) Preuve de la sécurité RLS ..... 8
3) Jalon 3 : ..... 8
3.1) Gestion des dépenses ..... 8
3.2) Ecran soldes et validation de jeu de test ..... 9
4) Jalon 4 : ..... 10
4.1) Tâches ménagères et rotation des tâches ..... 10
4.2) Liste des courses et Chat ..... 10
5) Jalon 5 : ..... 12
5.1) 5 Test automatisés ..... 12
5.2) Historique Git ..... 12
Journal de bord : JOURNAL.MD ..... 13
Décisions : DECISIONS.MD ..... 14
Conception : CONCEPTION.MD ..... 15

# 1) Jalon 1 :

## 1.1) Questions :

**1. Comment représenter « un utilisateur appartient à une coloc, avec un rôle » ?**

On a créé une table intermédiaire appelée **membres**. Elle relie l'identifiant de l'utilisateur (user_id) et celui de la colocation (coloc_id), avec une colonne role qui

contient soit 'admin' soit 'membre'.
Cela permet à un utilisateur de faire partie de plusieurs colocations en même temps.

**2. Comment représenter une dépense et sa répartition entre plusieurs personnes ? (Une seule table suffit-elle ?)**

Non, une seule table ne suffit pas. On a donc séparé les informations en 2 tables :

- La table **depenses** : pour stocker qui a payé, le montant total, le titre et la date.

- La table **repartitions** : pour stocker la part de chaque colocataire (une ligne par personne avec son montant dû).
Cela permet de partager une facture facilement, peu importe si les parts sont égales ou non.

**3. Stocke-t-on les montants en décimal ou en centimes entiers ? Pourquoi ?**

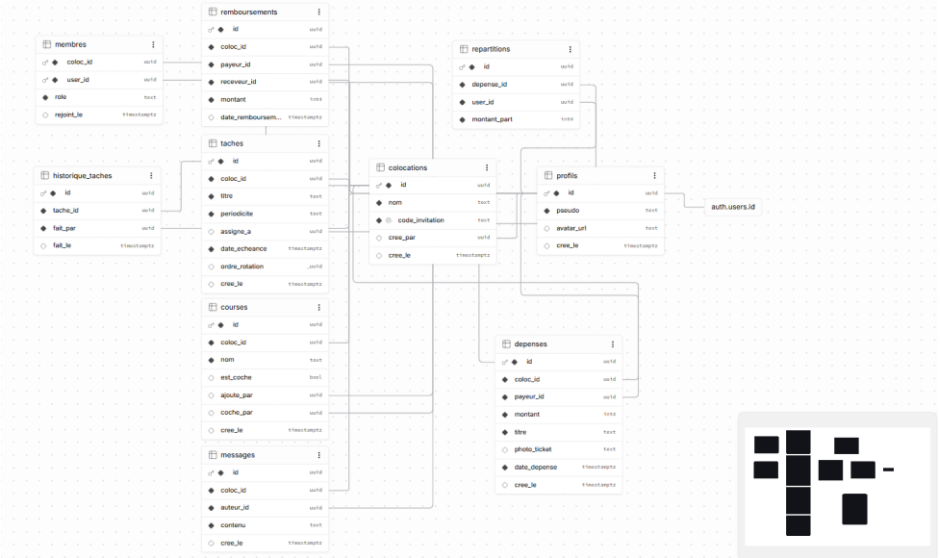
On stocke tous les montants en centimes entiers et non en float car les nombres à virgules float ont des petits bugs d'arrondis, du coup en utilisant des centimes entiers, les additions n'ont aucun de ces soucis d'arrondis et on n'a qu'à diviser par 100 du côté Flutter pour afficher le prix en euros

**4. Où stocker la rotation des tâches ?**

On stocke la rotation directement dans la table **taches** grâce à deux colonnes :
- **ordre_rotation** : c'est la liste ordonnée des colocataires qui doivent faire la tâche à tour de rôle.
- **assigne_a** : le colocataire dont c'est le tour actuellement.
Dès qu'un colocataire clique sur le bouton « Fait », l'application passe automatiquement la tâche au colocataire suivant dans la liste.

## 1.2) Diagramme BDD + Script SQL

**Diagramme BDD (Database Supabase) :**



**Script SQL :**

### 1. Table des profils utilisateurs

```
CREATE TABLE public.profils (
    id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    pseudo VARCHAR(100) NOT NULL,
    avatar_url TEXT
);
```

### 2. Table des colocations

```
CREATE TABLE public.colocations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nom VARCHAR(100) NOT NULL,
    code_invitation VARCHAR(10) UNIQUE NOT NULL,
    cree_par UUID REFERENCES public.profils(id)
);
```

### 3. Table des membres

```
CREATE TABLE public.membres (
    coloc_id UUID REFERENCES public.colocations(id) ON DELETE CASCADE,
    user_id UUID REFERENCES public.profils(id) ON DELETE CASCADE,
    role VARCHAR(20) DEFAULT 'membre',
    PRIMARY KEY (coloc_id, user_id)
);
```

### 4. Table des dépenses

```
CREATE TABLE public.depenses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    coloc_id UUID REFERENCES public.colocations(id) ON DELETE CASCADE,
    payeur_id UUID REFERENCES public.profils(id),
    montant INT NOT NULL,
    titre VARCHAR(255) NOT NULL,
    photo_ticket TEXT,
    date_depense TIMESTAMP DEFAULT now()
);
```

### 5. Table des répartitions

```
CREATE TABLE public.repartitions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    depense_id UUID REFERENCES public.depenses(id) ON DELETE CASCADE,
    user_id UUID REFERENCES public.profils(id),
    montant_part INT NOT NULL
);
```

### 6. Table des remboursements

```
CREATE TABLE public.remboursements (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    coloc_id UUID REFERENCES public.colocations(id) ON DELETE CASCADE,
    payeur_id UUID REFERENCES public.profils(id),
    receveur_id UUID REFERENCES public.profils(id),
    montant INT NOT NULL,
    date_remboursement TIMESTAMP DEFAULT now()
);
```

### 7. Table des tâches ménagères

```
CREATE TABLE public.taches (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    coloc_id UUID REFERENCES public.colocations(id) ON DELETE CASCADE,
    titre VARCHAR(100) NOT NULL,
    periodicite VARCHAR(50) NOT NULL,
    assigne_a UUID REFERENCES public.profils(id),
    date_echeance TIMESTAMP DEFAULT now()
);
```

### 8. Table de l'historique des tâches

```
CREATE TABLE public.historique_taches (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tache_id UUID REFERENCES public.taches(id) ON DELETE CASCADE,
    fait_par UUID REFERENCES public.profils(id),
    fait_le TIMESTAMP DEFAULT now()
);
```

### 9. Table des listes de courses

```
CREATE TABLE public.courses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    coloc_id UUID REFERENCES public.colocations(id) ON DELETE CASCADE,
    nom VARCHAR(255) NOT NULL,
    est_coche BOOLEAN DEFAULT false,
    ajoute_par UUID REFERENCES public.profils(id),
    coche_par UUID REFERENCES public.profils(id)
);
```

### 10. Table des messages

```sql
CREATE TABLE public.messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    coloc_id UUID REFERENCES public.colocations(id) ON DELETE CASCADE,
    auteur_id UUID REFERENCES public.profils(id),
    contenu TEXT NOT NULL,
    cree_le TIMESTAMP DEFAULT now()
);
```

### 11. Activation de la sécurité (ROW LEVEL SECURITY)

```sql
ALTER TABLE public.profils ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.colocations ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.membres ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.depenses ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.repartitions ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.remboursements ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.taches ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.historique_taches ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.courses ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.messages ENABLE ROW LEVEL SECURITY;
```

## 1.3) Matrice des autorisations (RLS) et découpage des tâches

### Matrice des autorisations :

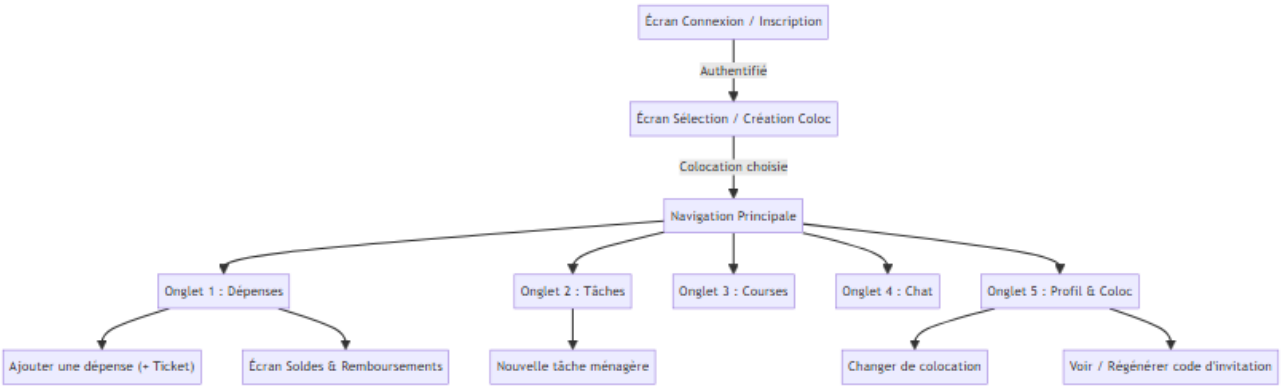
| Table | Action | Non-membre | Membre ordinaire | Admin coloc | Auteur / Payeur |
|---|---|---|---|---|---|
| **profils** | SELECT | Non | Oui | Oui | Oui |
| **profils** | UPDATE | Non | Non | Non | Oui (soi-même) |
| **colocations** | SELECT | Non | Oui (ses colocs) | Oui (ses colocs) | Oui |
| **colocations** | INSERT | Oui (création) | Oui | Oui | Oui |
| **membres** | SELECT | Non | Oui (sa coloc) | Oui (sa coloc) | Oui |
| **membres** | INSERT / DELETE | Non | Non | Oui (via RPC) | Oui (quitter via RPC) |
| **depenses** | SELECT | Non | Oui | Oui | Oui |
| **depenses** | INSERT | Non | Oui (si payeur) | Oui (si payeur) | Oui |
| **depenses** | UPDATE / DELETE | Non | Non | Oui | Oui |
| **repartitions** | SELECT / INSERT | Non | Oui | Oui | Oui |
| **remboursements** | SELECT / INSERT | Non | Oui | Oui | Oui (payeur) |
| **taches** | SELECT / INSERT / UPDATE | Non | Oui | Oui | Oui |
| **taches** | DELETE | Non | Non | Oui | Non |
| **courses** | SELECT / INSERT / UPDATE / DELETE | Non | Oui | Oui | Oui |
| **messages** | SELECT / INSERT | Non | Oui | Oui | Oui (auteur) |

**Découpage des tâches :**

Etant donné que je suis seul sur le projet, c'est moi qui gère toutes les tâches.

## 1.4) Graphe de navigation et maquettes papiers

**Graphe de navigation :**

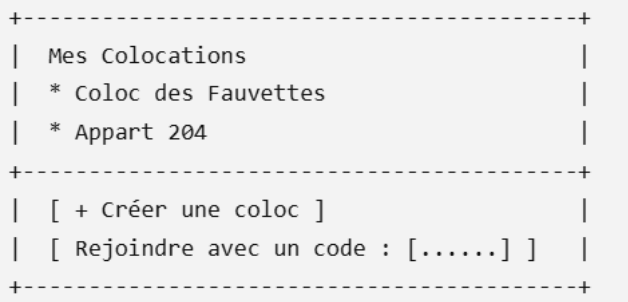



**Maquettes papier :**

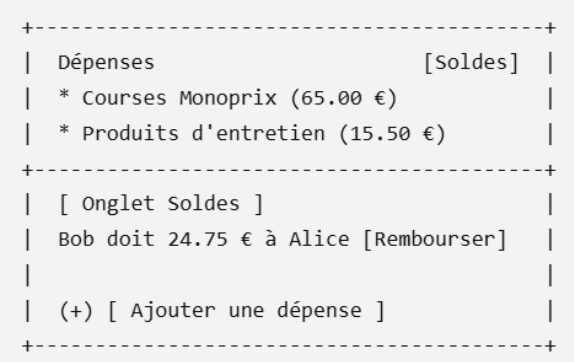
**1. Authentification**

```
+-----------------------------------------+
|                 COLLOC'                 |
|       Gérez votre vie en communauté     |
|                                         |
|  [ Inscription ]   |  [ Connexion ]     |
|                                         |
|  Email :        [ exemple@email.com   ] |
|  Mot de passe : [ **********          ] |
|                                         |
|           [ SE CONNECTER ]              |
+-----------------------------------------+
```

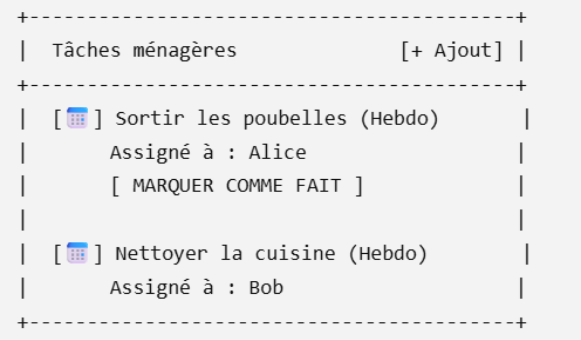
**2. Sélection / Création de collocation**



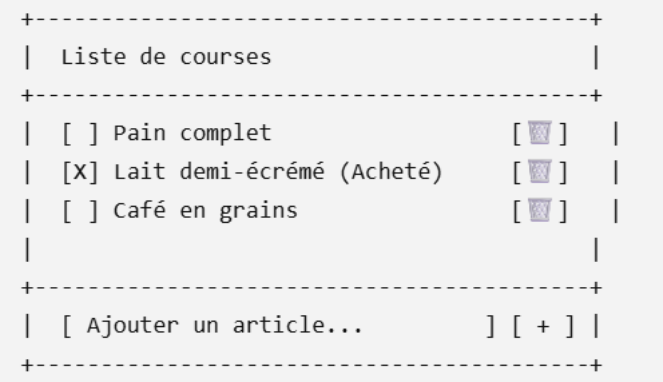

### 3. Dépenses et soldes




### 4. Tâches ménagères et rotations




### 5. Liste de courses partagées



# 2) Jalon 2 :

## 2.1) Authentification et sélection de la colocation

**Page d'authentification :**




**Page de sélection de la colocation :**




## 2.2) Preuve de la sécurité RLS

Le résultat du tableau est vide de qui signifie que la sécurité RLS fonctionne bel et bien :

```
PS C:\xampp\htdocs\colloc_app> # Tentative d'intrusion : Alice tente de lire les depenses de la colocation Appart 404
PS C:\xampp\htdocs\colloc_app> curl.exe -s -i -X GET "https://xnhbtdldlaiszflmucpi.supabase.co/rest/v1/depenses?
coloc_id=eq.2a514d0a-17a5-4517-a873-ac632c082adf" `
  -H "apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." `
  -H "Authorization: Bearer $token_alice"

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 2
Server: cloudflare

[]
```

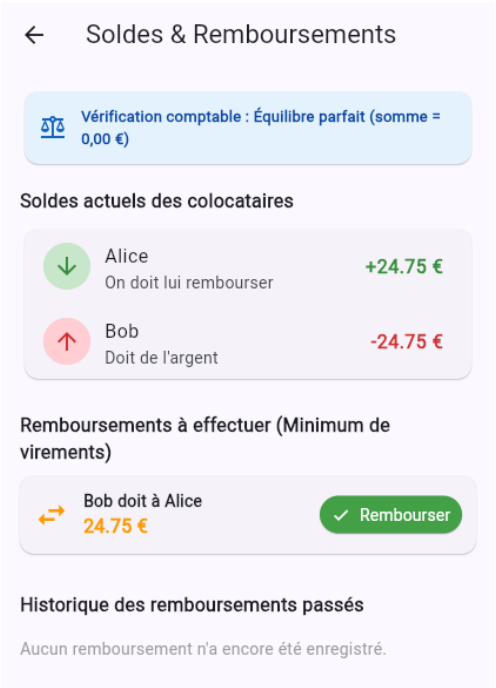
# 3) Jalon 3 :

## 3.1) Gestion des dépenses

Chaque dépense comporte un titre, un payeur, un montant en centimes, une date et la photo du ticket de caisse envoyée sur Supabase Storage (bucket tickets). La répartition peut se faire à parts égales ou personnalisées.




## 3.2) Ecran soldes et validation de jeu de test




Le calcul des soldes nets est réalisé en Dart (Solde = Payé − Dû + Remboursé − Reçu). L'algorithme calcule ensuite le minimum de virements nécessaires pour solder les dettes.

**Validation du jeu de test (4 personnes, 5 dépenses inégales) :**

- Alice : +14,50 €
- Bob : -20,00 €
- Charlie : -39,50 €
- David : +45,00 €

Somme des soldes = **0 euros**.
Les dettes sont résolues en seulement 3 virements optimisés.

# 4) Jalon 4 :

## 4.1) Tâches ménagères et rotation des tâches

Les tâches possèdent une périodicité (quotidienne ou hebdomadaire). L'application indique en rouge si une tâche est en retard. Lorsqu'un colocataire clique sur "Fait", la tâche passe automatiquement au colocataire suivant dans la liste ordonnée et l'action est enregistrée dans l'historique.




## 4.2) Liste des courses et Chat

Les listes de courses et les messages du chat utilisent la technologie temps réel de Supabase (supabase_realtime). Toute modification (article ajouté, coché, supprimé ou nouveau message) apparaît instantanément sur les téléphones des autres colocataires sans rafraîchir l'écran.

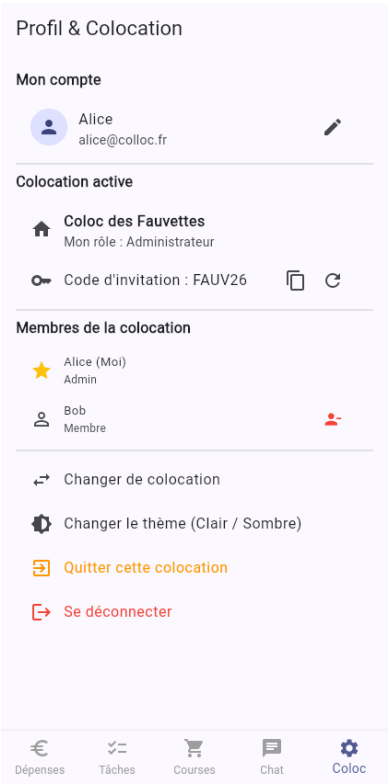
**Liste de courses :**



## Chat en direct :




## Profil coloc :



# 5) Jalon 5 :

## 5.1) 5 Test automatisés

Pour valider la fiabilité de l'application, nous avons écrit et exécuté une suite de 5 tests automatisés avec la commande flutter test. **1**00% des tests sont validés :

```
00:00 +0: loading C:/xampp/htdocs/colloc_app/test/arrondis_modulo_test.dart
00:00 +0: C:/xampp/htdocs/colloc_app/test/arrondis_modulo_test.dart: Test unitaire : Gestion des arrondis et repartition
 exacte du centime manquant
00:00 +1: C:/xampp/htdocs/colloc_app/test/remboursement_test.dart: Test unitaire : Impact dun remboursement sur les sold
es
00:00 +2: C:/xampp/htdocs/colloc_app/test/soldes_jalon3_test.dart: Jalon 3 : Jeu de test impose 4 personnes et 5 depense
s avec parts inegales
00:00 +3: C:/xampp/htdocs/colloc_app/test/widget_test.dart: Calcul des soldes et équilibre parfait (somme = 0)
00:00 +4: C:/xampp/htdocs/colloc_app/test/widget_authentification_test.dart: (setUpAll)
00:01 +4: C:/xampp/htdocs/colloc_app/test/widget_authentification_test.dart: Test de widget : Formulaire de connexion et
 validation des champs
00:02 +5: C:/xampp/htdocs/colloc_app/test/widget_authentification_test.dart: (tearDownAll)
00:02 +5: All tests passed!
```

## 5.2) Historique Git

Le projet a été versionné sous Git avec des commits réguliers retraçant chaque fonctionnalité et jalon :

```
C:\xampp\htdocs\colloc_app>git log
commit cfeef3f3a41eb765f5abe3e0298d65d6b817ab4a (HEAD -> master)
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 11:26:40 2026 +0200

    Jalon 5: Mode expanded pour conserver l'historique complet de chaque test sans fermeture

commit d630f347fd9cdae8c1c9d76e223941d8de9da245
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 11:21:15 2026 +0200

    Jalon 5: Ajout du script de lancement des tests et capture reelle de flutter test

commit a3bb85e72d7245e806f2450e09355c52a1f0e825
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 11:01:18 2026 +0200

    Jalon 2: Mise a jour capture curl RLS avec scenario Alice et Appart 404

commit 2431f0c3f0cabe792908f5ed8e05db7347eb663f
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 10:58:39 2026 +0200

    Jalon 2: Capture brute de la requete curl et de la reponse RLS vide []

commit be7a8216176e4dc2d10bb956b0790c768415de3f
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 10:54:00 2026 +0200

    Jalon 2: Script et capture de demonstration du test d'intrusion RLS

commit 51f0c150fe29b082754e9f082d98c84f545ae7f1
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 10:12:07 2026 +0200

    Jalon 1: Organisation explicite des 4 livrables de conception (schema, maquettes, matrice RLS, repartition des taches)

commit f7c68a84685b2ab40ddc024668a1328c5ecc0783
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 10:07:22 2026 +0200

    Jalon 5: Captures d'ecran reelles de l'application, documentation complete et journal de bord

commit 7bf13de91c2eac819dc62b6c14f4a190b4012700
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 09:54:35 2026 +0200

    Jalon 4 et 5: Taches menageres avec rotation automatique, courses et chat Realtime, suite de tests de widgets

commit fb42a9bf278a0d5b777d33b50012bef013d9e0c9
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 09:54:28 2026 +0200

    Jalon 3: Gestion des depenses, photo ticket, calcul des soldes nets et simplification gloutonne des dettes

commit 22ba2aaa1c81a3c4e507911eaf3b394f606d309d
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 09:54:22 2026 +0200

    Jalon 2: Authentification Supabase, creation et adhesion par code d'invitation avec RLS

commit 53cf6563964787ffdb6e1481c755c30bea46c94b
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 09:54:16 2026 +0200

    Jalon 2: Modeles de donnees en francais (Profil, Colocation, Membre, Depense, Repartition, Remboursement, Tache, Course, Message)

commit e097bc1aa8ff4aef620d3150ad3e2f0fdb51b923
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 09:54:01 2026 +0200

    Jalon 2: Initialisation du projet Flutter et ajout des dependances

commit 39e1e5618648062436536e0eaf37946d3b0580f9
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 09:32:32 2026 +0200

    Jalon 1: Conception de la base de donnees, RLS et architecture
(END)
Author: Ewan Alexandre <ewan.alexandre@sio.local>
Date:   Fri Oct 9 09:54:28 2026 +0200

    Jalon 3: Gestion des depenses, photo ticket, calcul des soldes nets et simplification gloutonne des dettes
```

# Journal de bord : JOURNAL.MD

## Séance 1 :

```
## Séance 1 : Conception et base de données (Jalon 1)

### 1. Ce que j'ai fait
- J'ai lu le sujet du projet et j'ai réfléchi à la base de données.
- J'ai créé les 10 tables sur Supabase en français (`colocations`, `membres`, `depenses`, `repartitions`, `remboursements`, `taches`, `historique_taches`, `courses`, `messages`, `profils`).
- J'ai écrit le script SQL `schema.sql` pour tout créer proprement avec les contraintes et les clés étrangères.
- J'ai choisi de stocker tous les montants en centimes d'euros pour pas avoir de soucis avec les virgules.
- J'ai fait les croquis sur papier des 5 écrans principaux et j'ai répondu aux 4 questions demandées par le prof.

### 2. Mon problème
- Quand j'ai voulu mettre les règles de sécurité RLS sur la table `membres`, ça tournait en boucle à l'infini parce que la règle essayait de lire la table `membres` pour savoir si on avait le droit de lire `membres`.
- En plus pour rejoindre une coloc avec un code, comme on n'est pas encore membre de la coloc, le RLS bloquait la recherche du code.

### 3. Ce que j'ai fait pour régler ça
- J'ai créé deux petites fonctions en SQL (`est_membre` et `rejoindre_colocation`) avec l'option `SECURITY DEFINER`. Comme ça la base de données fait la vérification en interne sans bloquer sur la sécurité.
```

## Séance 2 :

```
## Séance 2 : Début de Flutter et connexion (Jalon 2)

### 1. Ce que j'ai fait
- J'ai créé le projet Flutter avec `flutter create` et j'ai installé le package Supabase.
- J'ai rangé mon code en 4 dossiers simples : `modeles`, `services`, `ecrans`, et `widgets`.
- J'ai fait l'écran de connexion / inscription et l'écran pour choisir sa colocation ou en rejoindre une avec un code d'invitation.
- J'ai testé avec le compte d'Alice et celui de Bob.

### 2. Mon problème
- Il fallait prouver pour le prof que la sécurité RLS marche bien et qu'on ne peut pas aller fouiller dans les données d'une autre colocation.

### 3. Ce que j'ai fait pour régler ça
- J'ai fait une requête directe en ligne de commande avec `curl`. J'ai pris le jeton de connexion de mon utilisateur et j'ai essayé de demander les dépenses d'une autre colocation (*Appart 404*).
- Le serveur a renvoyé `HTTP 200` avec `[]` (un tableau vide), ce qui prouve bien que la base PostgreSQL a bloqué les données et ne renvoie rien.

---
```

## Séance 3 :

```
## Séance 3 : Gestion de l'argent et des dépenses (Jalon 3)

### 1. Ce que j'ai fait
- J'ai codé la page qui liste les dépenses et le formulaire pour en ajouter une avec le montant, le titre et qui a payé.
- J'ai créé la page des soldes pour voir qui doit de l'argent et à qui.
- J'ai codé l'algorithme qui simplifie les remboursements pour faire le moins de virements possible entre les colocs.
- J'ai testé avec l'exemple imposé du sujet (4 personnes et 5 dépenses inégales) pour vérifier que la somme de tous les soldes fait bien 0,00 €.

### 2. Mon problème
- Quand on divise par exemple 10 € en 3 parts égales, ça donne 3,33 € chacun. Le souci c'est que 3,33 x 3 = 9,99 €, donc il manquait 1 centime pour faire les 10 € au total.

### 3. Ce que j'ai fait pour régler ça
- J'ai mis une règle dans le calcul : l'application prend le centime qui reste et le donne au premier colocataire de la liste. Comme ça la somme fait toujours 10,00 € tout pile et les comptes sont justes.
```

## Séance 4 :

```
## Séance 4 : Tâches ménagères, courses et chat (Jalon 4)

### 1. Ce que j'ai fait
- J'ai fait l'écran des corvées et tâches ménagères avec le bouton "Fait" qui passe automatiquement la corvée au colocataire d'après.
- J'ai fait la liste de courses partagée où on peut cocher les articles qu'on a achetés.
- J'ai fait la messagerie instantanée (chat) entre les colocs.

### 2. Mon problème
- Il fallait que quand un colocataire coche un truc dans les courses ou envoie un message sur le chat, ça apparaisse tout de suite sur les téléphones des autres sans avoir besoin de recharger l'application à la main.

### 3. Ce que j'ai fait pour régler ça
- J'ai activé le Realtime sur Supabase. L'application écoute les tables en direct et met à jour l'écran automatiquement dès qu'il y a une nouveauté.
```

## Séance 5 :

```
## Séance 5 : Tests automatisés et finalisation (Jalon 5)

### 1. Ce que j'ai fait
- J'ai écrit les 5 tests automatisés avec Flutter (`flutter test`) pour tester les calculs de soldes, la gestion du centime manquant, les remboursements et la validation du formulaire de connexion.
- J'ai fait les captures d'écran de tous les écrans pour mon dossier Word.
- J'ai vérifié que tous mes commits Git sont bien faits et propres.

### 2. Mon problème
- Quand je lançais les tests, la fenêtre se fermait toute seule dès que c'était fini et en plus ça écrasait les lignes donc je ne voyais pas la liste complète de tous les tests qui réussissaient.

### 3. Ce que j'ai fait pour régler ça
- J'ai rajouté l'option `--reporter expanded` dans la commande pour que chaque test reste affiché sur sa propre ligne.
- J'ai créé un petit fichier `lancer_tests.bat` avec une pause à la fin pour pouvoir faire ma capture d'écran tranquillement.
```

# Décisions : DECISIONS.MD

```
### 1. Gestion de l'état dans l'application (State Management)
- **Options envisagées** : Tout faire avec `setState` partout, utiliser `Provider`, ou des packages plus poussés comme `Bloc` ou `Riverpod`.
- **Choix** : J'ai choisi `Provider` (avec `ChangeNotifier`), et juste un peu de `setState` classique dans les formulaires pour les champs locaux.
- **Raison** : Bloc et Riverpod sont beaucoup trop complexes pour mon niveau, il y a trop de fichiers et de notions compliquées. `Provider` est plus facile à comprendre, on l'a vu en cours, et ça permet de rafraîchir l'écran simplement dès que les données de la colocation changent.

---

### 2. Organisation des dossiers du projet
- **Options envisagées** : Tout mettre dans un seul dossier `lib`, faire des dossiers par fonctionnalité, ou séparer par rôle en français.
- **Choix** : J'ai séparé le code en 4 dossiers simples en français : `modeles`, `services`, `ecrans`, et `widgets`.
- **Raison** : C'est le découpage classique qu'on apprend en BTS (les données dans `modeles`, la communication avec Supabase dans `services`, et les vues dans `ecrans`). C'est propre, je sais directement où chercher quand je veux modifier quelque chose et je ne me perds pas.

---

### 3. Comment lier un utilisateur à une colocation avec son rôle
- **Options envisagées** : Mettre l'id de la colocation directement dans le profil de l'utilisateur, ou créer une table de liaison `membres`.
- **Choix** : J'ai créé une table à part `membres(coloc_id, user_id, role)`.
- **Raison** : Si j'avais mis la coloc directement dans le profil, l'utilisateur ne pourrait appartenir qu'à une seule colocation à la fois. Comme le sujet demande de pouvoir être dans plusieurs colocations en même temps et de passer de l'une à l'autre, une table de liaison était obligatoire.

---

### 4. Modèle pour une dépense et sa répartition
- **Options envisagées** : Mettre tout dans une seule table `depenses` avec du JSON, ou séparer en deux tables distinctes.
- **Choix** : Deux tables séparées : `depenses` (qui a payé, la date, le montant total) et `repartitions` (une ligne par colocataire concerné avec sa part).
- **Raison** : Une seule table aurait été trop compliquée à gérer dès que les parts sont inégales (par exemple si un coloc ne participe pas à un achat). Avec deux tables, c'est propre, conforme aux cours de base de données, et on peut facilement vérifier que la somme des parts est bien égale au total de la dépense.

---

### 5. Format de stockage de l'argent (nombres à virgule ou centimes ?)
- **Options envisagées** : Mettre des nombres à virgule (`FLOAT`), ou tout stocker sous forme de centimes entiers (`INT`).
- **Choix** : J'ai tout stocké en centimes entiers avec le type `INT` (par exemple 12,50 € devient 1250).
- **Raison** : Les nombres à virgule en informatique créent des bugs d'arrondi connus (par exemple `0.1 + 0.2` qui donne `0.300000004`). En utilisant des centimes entiers, il n'y a aucun risque de perdre de l'argent dans les calculs. Pour l'affichage dans l'application, je divise juste par 100 à la fin.

---

### 6. Rotation des tâches ménagères
- **Options envisagées** : Créer un calendrier complexe avec plein de dates à l'avance, ou stocker la liste des personnes dans l'ordre.
- **Choix** : J'ai mis la liste des colocataires dans l'ordre dans la tâche, avec un champ pour savoir à qui c'est le tour actuellement.
- **Raison** : C'est le plus simple et le plus logique. Dès que la personne clique sur le bouton "Fait", l'application enregistre l'action dans un historique et passe automatiquement au colocataire suivant dans la liste.

---

### 7. Emplacement du calcul des soldes et des remboursements
- **Options envisagées** : Faire les calculs directement sur Supabase en SQL, ou faire le calcul dans le code Flutter en Dart.
- **Choix** : J'ai fait les calculs côté Flutter dans un fichier `service_soldes.dart`.
- **Raison** : L'algorithme qui simplifie les dettes (pour trouver le minimum de virements entre les colocs au lieu que tout le monde se rembourse dans tous les sens) était plus simple à coder et à tester en Dart. Ça permet aussi de faire tourner les tests automatisés sans avoir besoin de connexion réseau.

---

### 8. Sécurité et protection des données (RLS sur Supabase)
- **Options envisagées** : Filtrer les données uniquement dans Flutter, ou bloquer directement au niveau de la base de données avec le RLS.
- **Choix** : J'ai tout sécurisé en base avec les règles Row Level Security (RLS) de Supabase.
- **Raison** : Si on filtre seulement dans l'application, n'importe qui peut utiliser un outil comme `curl` pour récupérer les dépenses des autres colocations. Avec le RLS, c'est la base de données PostgreSQL qui refuse directement d'envoyer les données si l'utilisateur ne fait pas partie de la colocation.

---

### 9. Gestion du centime manquant lors des divisions
- **Options envisagées** : Laisser un total incomplet (par exemple 9,99 € au lieu de 10 €), ou ajouter le centime restant à quelqu'un.
- **Choix** : Le reste de la division (le centime manquant) est automatiquement donné au premier colocataire de la liste.
- **Raison** : 10 € divisés par 3 personnes, ça donne 3,33 € par personne, ce qui fait 9,99 € au total. Pour éviter qu'il manque 1 centime dans les comptes, le premier colocataire paie 3,34 € et les autres 3,33 €. Comme ça la somme fait toujours exactement 10,00 €.
```

# Conception : CONCEPTION.MD

```
# LIVRABLE 1 : Schéma de base de données & Questions à trancher

### A. Réponses aux 4 questions imposées

#### 1. Comment représenter « un utilisateur appartient à une coloc, avec un rôle » ?
- **Réponse :** Un utilisateur peut appartenir à plusieurs colocations à la fois et basculer entre elles. Une colocation possède plusieurs colocataires. Il s'agit d'une relation plusieurs-à-plusieurs ($N:M$).
- **Modélisation :** On utilise une table d'association `membres` :
  - `coloc_id` (UUID, clé étrangère vers `colocations`)
  - `user_id` (UUID, clé étrangère vers `profils`)
  - `role` (TEXT, contrainte `CHECK (role IN ('admin', 'membre'))`, défaut `'membre'`)
  - `rejoint_le` (TIMESTAMPTZ, date d'adhésion)
  - Clé primaire composite : `PRIMARY KEY (coloc_id, user_id)`

#### 2. Comment représenter une dépense et sa répartition entre plusieurs personnes ? (Une seule table suffit-elle ?)
- **Réponse :** Non, une seule table ne suffit pas. Une dépense est payée par une personne mais peut concerner 2, 3, 4 personnes ou toute la coloc, avec des parts qui peuvent être égales ou inégales.
- **Modélisation :** On sépare les données en 2 tables reliées par une clé étrangère :
  1. Table `depenses` : contient les informations globales (titre, payeur, montant total en centimes, date, photo du ticket).
  2. Table `repartitions` : contient une ligne par colocataire concerné avec son `montant_part` en centimes.

#### 3. Stocke-t-on les montants en décimal ou en centimes entiers ? Pourquoi ?
- **Réponse :** Nous stockons tous les montants sous forme d'**entiers (`INT`) représentant des centimes d'euros** (ex: 15,90 € = 1590 centimes).
- **Pourquoi ?** Les nombres à virgule flottante (`float`, `double`) provoquent des erreurs d'arrondis binaires en mémoire (ex: $0.1 + 0.2 = 0.30000000000000004$). En manipulant des entiers pour les centimes :
  - Aucun risque de perte de centimes en base ou dans les calculs.
  - La vérification comptable ($\sum \text{parts} = \text{total}$) est exacte à 100%.
  - Côté affichage Flutter, il suffit de diviser par 100 : `(montant / 100).toStringAsFixed(2) + ' €'`.

#### 4. Où stocker la rotation des tâches ?
- **Réponse :** La rotation est stockée directement dans la table `taches` avec :
  - `assigne_a` (UUID, pointe vers le membre actuellement en charge).
  - `ordre_rotation` (UUID[], tableau ordonné des identifiants des colocataires participant à la tâche).
  - Lorsqu'un membre valide la tâche, une entrée est archivée dans la table `historique_taches`, et le pointeur `assigne_a` passe automatiquement au colocataire suivant dans la liste ordonnée.
```

```
### B. Diagramme Relationnel (Mermaid ERD)

```mermaid
erDiagram
    PROFILS ||--o{ MEMBRES : "appartient"
    COLOCATIONS ||--o{ MEMBRES : "comprend"
    PROFILS ||--o{ DEPENSES : "paie"
    COLOCATIONS ||--o{ DEPENSES : "contient"
    DEPENSES ||--o{ REPARTITIONS : "se divise en"
    PROFILS ||--o{ REPARTITIONS : "doit une part"
    COLOCATIONS ||--o{ REMBOURSEMENTS : "enregistre"
    PROFILS ||--o{ REMBOURSEMENTS : "verse / recoit"
    COLOCATIONS ||--o{ TACHES : "possede"
    PROFILS ||--o{ TACHES : "est assigne a"
    TACHES ||--o{ HISTORIQUE_TACHES : "genere"
    COLOCATIONS ||--o{ COURSES : "comporte"
    COLOCATIONS ||--o{ MESSAGES : "echange"
```

### C. Détail des 10 tables SQL (100% en français)
Le script complet exécuté sur Supabase est disponible dans [`docs/schema.sql`](file:///C:/xampp/htdocs/colloc_app/docs/schema.sql) :
1. **`profils`** (`id` UUID PK, `pseudo` TEXT, `avatar_url` TEXT, `cree_le` TIMESTAMPTZ)
2. **`colocations`** (`id` UUID PK, `nom` TEXT, `code_invitation` TEXT UNIQUE, `cree_par` UUID FK, `cree_le` TIMESTAMPTZ)
3. **`membres`** (`coloc_id` UUID FK, `user_id` UUID FK, `role` TEXT CHECK, `rejoint_le` TIMESTAMPTZ)
4. **`depenses`** (`id` UUID PK, `coloc_id` UUID FK, `payeur_id` UUID FK, `montant` INT, `titre` TEXT, `photo_ticket` TEXT, `date_depense` TIMESTAMPTZ)
5. **`repartitions`** (`id` UUID PK, `depense_id` UUID FK, `user_id` UUID FK, `montant_part` INT)
6. **`remboursements`** (`id` UUID PK, `coloc_id` UUID FK, `payeur_id` UUID FK, `receveur_id` UUID FK, `montant` INT, `date_remboursement` TIMESTAMPTZ)
7. **`taches`** (`id` UUID PK, `coloc_id` UUID FK, `titre` TEXT, `periodicite` TEXT, `assigne_a` UUID FK, `date_echeance` TIMESTAMPTZ, `ordre_rotation` UUID[])
8. **`historique_taches`** (`id` UUID PK, `tache_id` UUID FK, `fait_par` UUID FK, `fait_le` TIMESTAMPTZ)
9. **`courses`** (`id` UUID PK, `coloc_id` UUID FK, `nom` TEXT, `est_coche` BOOLEAN, `ajoute_par` UUID FK, `coche_par` UUID FK)
10. **`messages`** (`id` UUID PK, `coloc_id` UUID FK, `auteur_id` UUID FK, `contenu` TEXT, `cree_le` TIMESTAMPTZ)
```

```
# LIVRABLE 2 : Maquettes de 5 écrans clés & Graphe de navigation

### A. Maquettes textuelles des 5 écrans clés (Wireframes papier)

#### Écran 1 : Connexion & Inscription
```text
+-----------------------------------------+
|                 COLLOC'                 |
|       Gérez votre vie en communauté     |
|                                         |
|  [ Inscription ]   |  [ Connexion ]     |
|                                         |
|  Email :        [ exemple@email.com   ] |
|  Mot de passe : [ **********          ] |
|  Pseudo (si inscription) : [ Ewan     ] |
|                                         |
|           [ SE CONNECTER ]              |
+-----------------------------------------+
```

#### Écran 2 : Sélection / Création / Adhésion de Colocation
```text
+-----------------------------------------+
|  Mes Colocations                  [+]   |
+-----------------------------------------+
|  * Coloc des Fauvettes (Actuelle - Admin)|
|  * Appart 204                           |
+-----------------------------------------+
|  Rejoindre une coloc avec un code :     |
|  [ Code à 6 lettres/chiffres ] [Rejoindre]|
|                                         |
|  Ou :                                   |
|  [ + Créer une nouvelle colocation ]    |
+-----------------------------------------+
```
```

#### Écran 3 : Dépenses & Soldes
```text
+-----------------------------------------+
|  Dépenses - Coloc des Fauvettes [Soldes] |
+-----------------------------------------+
|  (65€) Courses Monoprix        65.00 €   |
|        Payé par Alice le 09/10           |
|  --------------------------------------- |
|  (16€) Produits d'entretien    15.50 €   |
|        Payé par Bob le 09/10             |
+-----------------------------------------+
|  [ Écran Soldes accessible via le bouton ]|
|  * Alice : +24.75 € (On doit lui rembourser)
|  * Bob :   -24.75 € (Doit de l'argent)   |
|  Remboursement suggéré :                 |
|  Bob doit 24.75 € à Alice  [Rembourser]  |
|                                          |
|  (+) [ Ajouter une dépense ]             |
+-----------------------------------------+
```

#### Écran 4 : Tâches ménagères & Rotation
```text
+-----------------------------------------+
|  Tâches ménagères             [+ Ajout] |
+-----------------------------------------+
|  [🗑] Sortir les poubelles (Hebdo)       |
|       Assigné à : Alice                  |
|       Pour le : 16/10/2026               |
|       [ FAIT ] -> Fait tourner la tâche  |
|                                          |
|  [🗑] Nettoyer la cuisine (Hebdo)        |
|       Assigné à : Bob                    |
|       [ FAIT ]                           |
+-----------------------------------------+
```

#### Écran 5 : Courses partagées & Chat en direct
```text
+-----------------------------------------+
|  Courses [X]            |  Chat [ ]      |
+-----------------------------------------+
|  [ ] Pain complet                 [🗑]   |
|  [X] Lait demi-écrémé (Acheté)    [🗑]   |
|  [ ] Café en grains               [🗑]   |
|  [ Ajouter un article...     ] [+]       |
+-----------------------------------------+
|  (Si onglet Chat sélectionné) :          |
|  Alice: Salut Bob ! Bienvenue !          |
|  Bob: Merci Alice, l'appli marche super !|
|  [ Écrire un message...       ] [Envoyer]|
+-----------------------------------------+
```

### B. Graphe de navigation

```mermaid
flowchart TD
    Login[Écran Connexion / Inscription] -->|Authentifié| ChoixColoc[Écran Sélection / Création Coloc]
    ChoixColoc -->|Colocation sélectionnée| Accueil[Navigation Principale]

    Accueil --> E1[Onglet 1 : Dépenses]
    Accueil --> E2[Onglet 2 : Tâches]
    Accueil --> E3[Onglet 3 : Courses]
    Accueil --> E4[Onglet 4 : Chat]
    Accueil --> E5[Onglet 5 : Profil & Coloc]

    E1 --> AjoutDepense[Ajouter une Dépense avec Ticket]
    E1 --> VueSoldes[Écran Soldes & Remboursements]
    E2 --> AjoutTache[Dialogue Nouvelle Tâche]
    E5 --> SwitchColoc[Changer de Colocation]
    E5 --> InviteCode[Voir / Régénérer Code d'invitation]
```

---

# LIVRABLE 3 : Matrice d'autorisations (Sécurité RLS)

Chaque table est protégée exclusivement par **Row Level Security (RLS)** dans Supabase. Aucune sécurité n'est déléguée au client mobile.

| Table | Action | Non-membre | Membre ordinaire | Admin coloc | Auteur / Payeur |
| --- | --- | --- | --- | --- | --- |
| **`profils`** | SELECT | Non | Oui | Oui | Oui |
| | UPDATE | Non | Non | Non | Oui (soi-même) |
| **`colocations`** | SELECT | Non | Oui (ses colocs) | Oui (ses colocs) | Oui |
| | INSERT | Oui (création) | Oui (création) | Oui | Oui |
| | UPDATE (code) | Non | Non | Oui (RPC) | Non |
| **`membres`** | SELECT | Non | Oui (de sa coloc) | Oui (de sa coloc) | Oui |
| | INSERT / DELETE | Non | Non | Oui (RPC) | Oui (quitter via RPC) |
| **`depenses`** | SELECT | Non | Oui | Oui | Oui |
| | INSERT | Non | Oui (si payeur) | Oui (si payeur) | Oui |
| | UPDATE / DELETE | Non | Non | Oui | Oui |
| **`repartitions`** | SELECT | Non | Oui | Oui | Oui |
| | INSERT | Non | Oui | Oui | Oui |
| **`remboursements`** | SELECT | Non | Oui | Oui | Oui |
| | INSERT | Non | Oui (si payeur) | Oui (si payeur) | Oui |
| **`taches`** | SELECT | Non | Oui | Oui | Oui |
| | INSERT / UPDATE | Non | Oui | Oui | Oui |
| | DELETE | Non | Non | Oui | Non |
| **`courses`** | SELECT / INSERT / UPDATE / DELETE | Non | Oui | Oui | Oui |
| **`messages`** | SELECT | Non | Oui | Oui | Oui |
| | INSERT | Non | Oui | Oui | Oui (auteur) |
| | DELETE | Non | Non | Oui | Oui (auteur) |

---

```
# LIVRABLE 4 : Découpage en tâches réparties entre les membres

Pour un projet en binôme (ou trinôme) de BTS SIO :

| Étudiant / Membre | Responsabilités & Modules assignés | Livrables associés |
| :--- | :--- | :--- |
| **Étudiant A (Lead Back-end & Sécurité)** | - Modélisation de la base SQL et script `schema.sql`<br>- Configuration des règles RLS et fonctions `SECURITY DEFINER`<br>- Service d'authentification (`service_auth.dart`)<br>- Gestion et adhésion colocation (`service_coloc.dart`)<br>- Tests d'intrusion curl et vérification des policies | **Jalon 1 & Jalon 2** |
| **Étudiant B (Gestion financière & Calculs)** | - Module Dépenses et formulaires (`ecran_ajout_depense.dart`)<br>- Upload des tickets dans Supabase Storage (`bucket tickets`)<br>- Implémentation de l'algorithme glouton des soldes (`service_soldes.dart`)<br>- Gestion des remboursements et de l'arrondi au centime<br>- Rédaction des tests unitaires comptables | **Jalon 3 & Jalon 5** |
| **Travail commun (Temps réel & Interface)** | - Module Tâches ménagères avec rotation (`service_taches.dart`)<br>- Liste de courses collaborative et Chat en Realtime (`courses`, `messages`)<br>- Interface mobile minimale et intégration du thème clair/sombre<br>- Rédaction du journal de bord (`JOURNAL.md`) et des décisions (`DECISIONS.md`) | **Jalon 4 & Finalisation** |
```